



UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

GENERACIÓN DE MALLAS POLIGONALES A PARTIR DE CAVIDADES

MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL EN COMPUTACIÓN

NICOLÁS ESCOBAR ZARZAR

PROFESORA GUÍA:
Nancy Hitschfeld K.

PROFESOR CO-GUÍA:
Sergio Salinas

MIEMBROS DE LA COMISIÓN:
Éric Tanter
Mauricio Cerda

Este trabajo ha sido parcialmente financiado por Fondecyt 1241596

SANTIAGO DE CHILE
2025

GENERACIÓN DE MALLAS POLIGONALES A PARTIR DE CAVIDADES

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Tabla de Contenido

1. Introducción	1
1.1. Introducción y Motivación	1
1.2. Objetivos	4
1.2.1. Objetivo General	4
1.2.2. Objetivos Específicos	4
1.3. Contenido de la memoria	4
2. Marco Teórico	6
2.1. Conceptos relevantes	6
2.1.1. Triangulación	6
2.1.2. Planar Straight Line Graph (PLSG)	6
2.1.3. Triangulación de Delaunay	6
2.1.4. Cápsula convexa	6
2.1.5. Diagrama de voronoi	7
2.1.6. Malla poligonal	8
2.1.7. Mallas triangulares	9
2.1.8. Mallas de polígonos arbitrarios	9
2.1.9. <i>Finite Element Method</i> (FEM) y <i>Virtual Element Method</i> (VEM)	9
2.1.10. Polígono simple y no simple	9
2.1.11. Cavidad	10
2.2. Estado del arte	10
2.2.1. Triangle	10
2.2.2. Detri2	14
2.2.3. Polylla	14
2.3. CGAL	17
3. Problema	18
4. Conclusión	19

Bibliografía	20
A. Anexo	22

Índice de Tablas

Índice de Ilustraciones

Figura 1	Triangulación de Delaunay con circuncírculos compuesta por 10 vértices. . . .	2
Figura 2	Selección de triángulos para formar un polígono a partir de una cavidad. . . .	3
Figura 3	Polígonos resultantes de las cavidades marcados en azul con los lados discontinuos borrados	3
Figura 4	a) Triangulación de Delaunay y Diagrama de Voronoi superpuesto, b) Triangulación de Delaunay, c) Diagrama de Voronoi [1]	7
Figura 5	Triangulación de Delaunay (en negro) junto a su diagrama de voronoi (en rojo). Fuente: Wikimedia[2]	7
Figura 6	Ejemplo de estructura <i>half-edge</i> [3]	8
Figura 7	Ejemplo de polígono no simple	10
Figura 8	PSLG de entrada una guitarra electrica como se ilustra en Triangle [4]	10
Figura 9	Triangulación del PSLG de la Figura 8 con segmentos originales faltantes [4]	11
Figura 10	Triangulación restringida del PSLG de la Figura 8 [4]	11
Figura 11	Triangulación restringida del PSLG de la Figura 8 con triangulos extra removidos [4]	12
Figura 12	Segmentos divididos según su círculo diametral [4]	13
Figura 13	Triángulo separado según su circuncentro [4]	13
Figura 14	Malla poligonal final resultante de aplicar el algoritmo Triangle [4]	13
Figura 15	Triangulación de Delaunay y su Diagrama de Voronoi dual en el software Detri2	14
Figura 16	Región de arista terminal. a) $Lepp(t_0)$ donde la arista roja es la arista terminal. b) Cuatro Lepps con la misma arista terminal: $Lepp(t_a)$, $Lepp(t_b)$, $Lepp(t_c)$, $Lepp(t_d)$. c) Región de arista terminal generada por la unión de los Lepp de b) [1]	15
Figura 17	a) Colección de vértices aleatorios, b) Triangulación de Delaunay donde las líneas sólidas son aristas frontera, las líneas punteadas negras son aristas internas y las aristas punteadas rojas son aristas terminales. c) Partición a partir de regiones de arista terminal [1]	16
Figura 18	Triangulación de Delaunay y su malla Polylla respectiva [5]	16

Capítulo 1

Introducción

1.1. Introducción y Motivación

Las mallas poligonales son estructuras fundamentales en el modelado geométrico y la simulación computacional. Se definen como colecciones de vértices, aristas y caras que, en conjunto, representan la superficie de un objeto, generalmente mediante una subdivisión en triángulos. Esta representación es ampliamente utilizada en áreas como el diseño asistido por computador (CAD), gráficos por computadora, ingeniería estructural, biomecánica y videojuegos. La eficiencia y calidad de las mallas generadas influye directamente en la precisión de las simulaciones y en el rendimiento computacional de los sistemas que las utilizan.

Uno de los problemas recurrentes en este ámbito es la generación automática de mallas poligonales a partir de un conjunto discreto de puntos en el plano, también conocido como problema de triangulación. Si bien existen algoritmos eficientes disponibles en herramientas como Triangle [4] o Detri2 [6] para formar una triangulación de Delaunay [7], la construcción de formas poligonales a partir de dichas triangulaciones sigue siendo un área de investigación activa. En particular, existe un interés creciente por encontrar métodos que generen mallas con mayor fidelidad geométrica, adaptabilidad local, y que mantengan cualidades deseables como ángulos adecuados y buena distribución de los elementos.

Una triangulación de Delaunay se define como tal si cumple con la propiedad de que para todos los triángulos de la triangulación, se cumple que el circuncírculo del triángulo solo contiene a los vértices de su triángulo respectivo y no los vértices de cualquier otro (ver Figura 1). Estas son de particular importancia porque maximizan el tamaño del ángulo más pequeño de la triangulación. Esta propiedad previene problemas de precisión que ocurren al tener ángulos muy agudos los cuales propician “casos degenerados” donde los puntos de un triángulo son interpretados como colineales lo cual puede provocar cálculos erróneos e incluso que los programas que trabajen con las mallas resultantes que no sean lo suficientemente robustos fallen por completo en el peor de los casos.

Este trabajo de memoria se enfoca en el desarrollo y análisis de una estrategia alternativa de generación de mallas en dos dimensiones, basada en el concepto de **cavidad** o *concavity* como se describe en el artículo Triangle [4]. La idea central es que, a partir de una triangulación de Delaunay (como la de la Figura 1), se seleccionan ciertos triángulos de la malla en un orden particular utilizando un criterio definido por el usuario. Luego, se calculan los circuncentros p_i de estos triángulos, y se identifican los conjuntos de triángulos de la malla cuyo circuncírculo contiene alguno de estos puntos por separado. La unión de las aristas del borde de estos triángulos vecinos para un punto p forma una **cavidad**, la cual define un nuevo polígono. Este procedimiento permite generar regiones poligonales que pueden servir como base para construir un nuevo tipo de mallas poligonales.

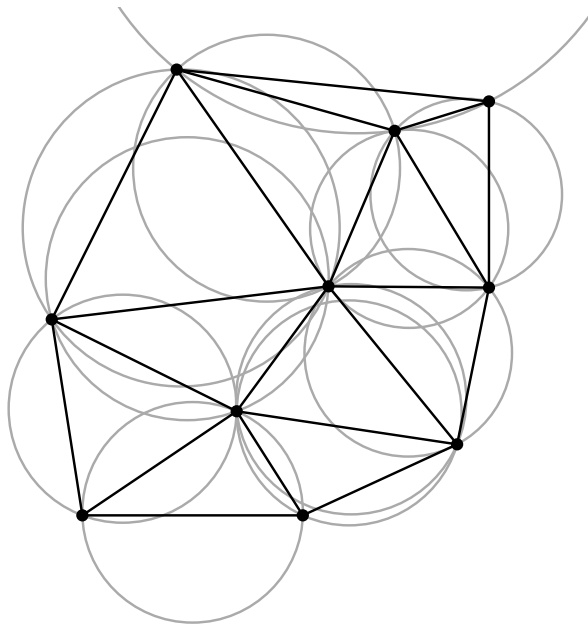


Figura 1: Triangulación de Delaunay con circuncírculos compuesta por 10 vértices.

A modo ilustrativo, en la Figura 2 se presenta una selección de triángulos a partir de la triangulación de la Figura 1.

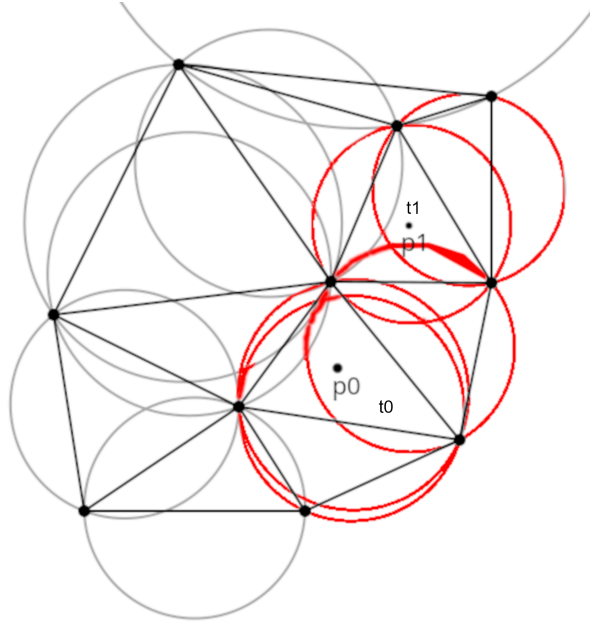


Figura 2: Selección de triángulos para formar un polígono a partir de una cavidad.

En este ejemplo, se seleccionaron dos triángulos t_0 y t_1 , cuyos respectivos circuncentros están representados por los puntos p_0 y p_1 . Estos puntos están contenidos en los circuncentros de varios triángulos vecinos, incluyendo aquellos que los generaron. La unión de las aristas de borde de los triángulos que contienen a p_0 y la unión de las aristas de borde de los triángulos que contienen a p_1 forman cada uno una cavidad, las cuales se pueden ver en la Figura 3.

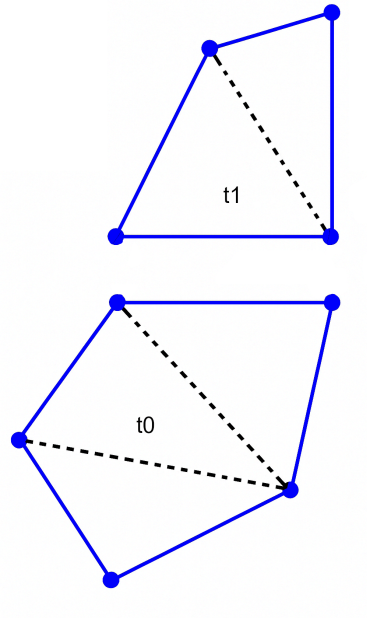


Figura 3: Polígonos resultantes de las cavidades marcados en azul con los lados discontinuos borrados

Este enfoque mediante cavidades no solo busca explorar una nueva forma de construir mallas, sino también comparar su rendimiento y características con métodos existentes. En particular, se contrastará esta técnica con el generador de mallas *Polylla* [1], un sistema moderno que emplea estructuras de datos avanzadas para lograr eficiencia y calidad en la generación de polígonos.

La necesidad de desarrollar nuevas estrategias para la generación de mallas responde a múltiples factores: mejorar la eficiencia de los algoritmos existentes, generar elementos con mejores propiedades geométricas, y adaptar las mallas a dominios complejos sin intervención manual.

1.2. Objetivos

1.2.1. Objetivo General

Diseñar e implementar un algoritmo que permita generar polígonos a partir de una *cavidad* desde una triangulación de Delaunay y comparar su desempeño con el generador de mallas Polylla en cuanto a tiempo, memoria, calidad de las mallas, número de vértices, aptitud para el VEM, entre otros criterios.

1.2.2. Objetivos Específicos

1. Investigar como funciona el algoritmo de mallas Polylla, sus estructuras de datos y métricas apropiadas para el VEM.
2. Definir escenarios de prueba para que el conjunto de polígonos que conforman la cavidad computada sean los correctos y generar un nuevo polígono a partir de ellos.
3. Comparar el desempeño del algoritmo basado en cavidades con el algoritmo de Polylla basado en regiones terminales en cuanto a eficiencia en tiempo y memoria.
4. Diseñar e implementar estrategias para seleccionar triángulos cuyo circuncentro se usará para definir cavidades.
5. Comparar la calidad de las mallas resultantes de ambos algoritmos bajo diversas métricas como calidad de la malla, número de aristas, cantidad de polígonos, etc.

1.3. Contenido de la memoria

Los capítulos siguientes de esta memoria abordan lo siguiente:

En el capítulo 2 se describen los conceptos necesarios para comprender este trabajo de memoria, junto con un resumen del estado del arte listando herramientas existentes. En el capítulo 3 se plantea el problema a resolver, se explica el funcionamiento del algoritmo a adaptar (Polylla[1]), su diseño, un breve análisis de aspectos a mejorar y el diseño del algoritmo basado en cavidades junto con una explicación del mismo.

En el capítulo 4 se muestra la implementación de la solución, su estructura a nivel de código, algoritmos concretos y validaciones. En el capítulo 5 se presentan los resultados (mallas poligonales) de la ejecución del algoritmo en diversas configuraciones, detallando

estadísticas obtenidas y tablas comparativas de calidad, tiempo y memoria tanto del algoritmo original, su reimplementación y el algoritmo de la solución.

Finalmente en el capítulo 6 se analizan los resultados obtenidos y un posible trabajo futuro.

Capítulo 2

Marco Teórico

2.1. Conceptos relevantes

A continuación se explican los conceptos relevantes utilizados a lo largo del trabajo de memoria.

2.1.1. Triangulación

Una triangulación es una forma de subdividir un objeto u espacio mediante el uso de triángulos, insertando puntos en su interior para formarlos de ser necesario. Una triangulación es un caso particular de una malla poligonal.

2.1.2. Planar Straight Line Graph (PLSG)

Un conjunto de vértices y segmentos que describen un polígono como el de la Figura 8. Los segmentos de un PLSG describen una forma o borde concreto que puede no ser convexa como la del ejemplo.

2.1.3. Triangulación de Delaunay

Una triangulación de Delaunay cumple la propiedad de que para todo triángulo que conforma la triangulación, su *circuncírculo*, es decir, el círculo único cuya circunferencia pasa por sus 3 vértices, solo contiene a los vértices del mismo triángulo y ningún otro punto, como en la Figura 1. Como se mencionó anteriormente, las triangulaciones de Delaunay son particularmente útiles debido a que maximizan el ángulo más pequeño de la triangulación, cuya utilidad será explicada más adelante. Cuando una triangulación de Delaunay se utiliza para subdividir un objeto con un borde concreto, como el de uno descrito por un PLSG, se dice que la triangulación de Delaunay es restringida, ya que debe incluir dichas aristas y no tener aristas fuera del borde. Esta distinción se hace porque típicamente una triangulación de Delaunay suele triangular conjuntos de puntos sin un borde definido inicialmente, el cual termina siendo la cápsula convexa del conjunto.

2.1.4. Cápsula convexa

Dado un conjunto de puntos, su cápsula convexa es el menor polígono convexo (en cuanto a superficie o volumen) que contiene todos los puntos en su interior.

2.1.5. Diagrama de voronoi

Un diagrama de Voronoi es una partición de un dominio P en regiones o “celdas” R_i , de las cuales cada una contiene un punto p_i llamado “semilla” y los puntos q_i contenidos en R_i cumplen que $\|q_i - p_i\| < \|q_i - p_j\| \forall p_i, p_j \in P, i \neq j$ donde p_j es la semilla de cualquier otra celda distinta a R_i . El diagrama de Voronoi también se le conoce como el *dual* de una triangulación de Delaunay, esto se debe a que, dada una triangulación de Delaunay, se puede obtener su diagrama de Voronoi equivalente si los circuncentros de los triángulos se convierten en puntos para las regiones de Voronoi (el circuncentro es el punto al centro del circuncírculo de un triángulo). Al unir los circuncentros mediante aristas, se forman las regiones del diagrama de Voronoi. En la Figura 4 y la Figura 5 se puede ver un ejemplo de una triangulación de Delaunay con su diagrama de Voronoi respectivo superpuesto.

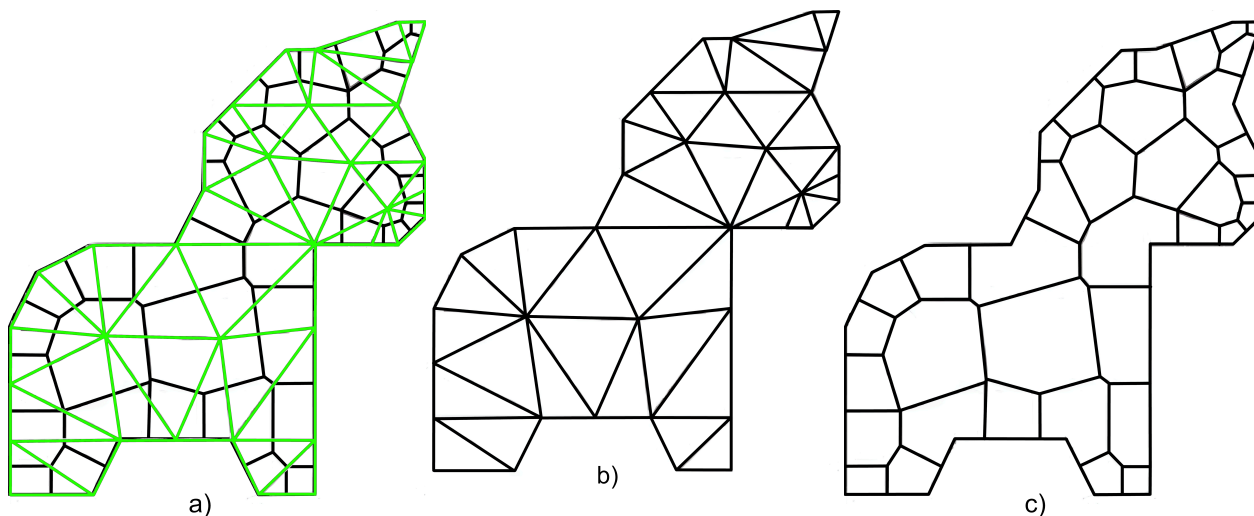


Figura 4: a) Triangulación de Delaunay y Diagrama de Voronoi superpuesto,
b) Triangulación de Delaunay,
c) Diagrama de Voronoi [1]

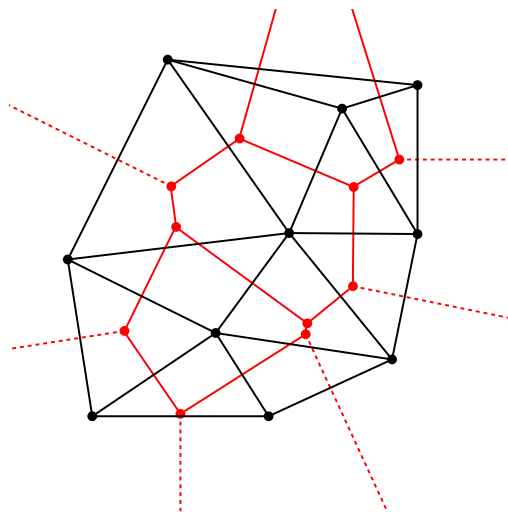


Figura 5: Triangulación de Delaunay (en negro) junto a su diagrama de voronoi (en rojo).
Fuente: Wikimedia[2]

El diagrama de Voronoi es otro caso particular de malla poligonal.

2.1.6. Malla poligonal

Una malla poligonal, o malla geométrica, es una forma de describir un objeto o un espacio como una colección de polígonos adyacentes. Estos polígonos se denotan según sus vértices y aristas que unen dichos vértices para formarlos (sus caras). Dichos polígonos pueden existir en un espacio en dos, tres o incluso más dimensiones dependiendo del caso de uso. Este trabajo de memoria solo se centrará en aplicaciones a mallas geométricas en 2D.

Una malla se puede representar de varias formas, siendo la más común una basada en caras, en que se guarda la información de los vértices que componen la malla y qué vértices forman cada cara, siendo las aristas guardadas de manera implícita en las caras. En la solución propuesta se hace uso de esta representación al recibir una malla como entrada contenida en uno o más archivos de texto, ya sea en formato `.node`, `.ele` y opcionalmente `.neigh` que guardan vértices, aristas (en forma de caras) e información de adyacencia respectivamente o en formato `.off` que guarda información de vértices y aristas, pero no de adyacencia. También se escribirán las mallas resultantes en formato `.off` o en el formato binario `.ale` utilizado principalmente por MATLAB para usar la malla en una solución numérica de una ecuación diferencial en derivadas parciales.

Además de esta representación, se hará uso de la estructura *Half Edge*, la cual guarda los vértices y divide las aristas de cada polígono en dos, una arista en sentido horario (abreviado como *CW* por *clockwise* del inglés) y otra en sentido antihorario (abreviado como *CCW* por *counter clockwise*). Por convención, una arista en sentido antihorario, se considera como una arista interna a un polígono y un polígono cualquiera se representa como el bucle completo que inicia desde una arista en sentido CCW y vuelve a la misma. Cada arista posee la siguiente información: un vértice de origen (*origin*), un vértice objetivo (*target*), su arista siguiente y anterior (según su orientación, llamadas también *next* y *prev* respectivamente), y su arista “gemela” (*twin*), la cual representa la misma arista en el sentido contrario. En la Figura 6 se puede ver un ejemplo de esta estructura.

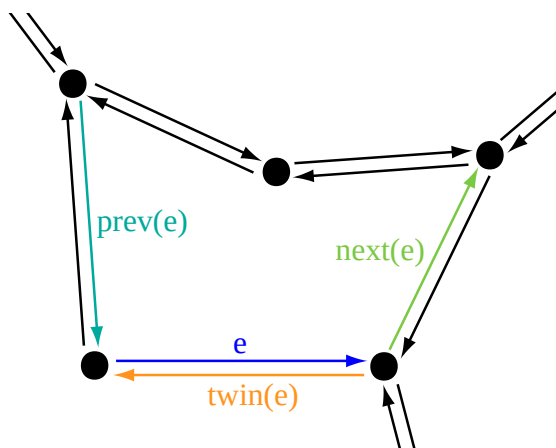


Figura 6: Ejemplo de estructura *half-edge*[3]

2.1.7. Mallas triangulares

Las mallas poligonales se suelen describir como conjuntos de triángulos, ya que al ser el polígono con menos aristas permite hacer una discretización, es decir, una división en partes concretas, del objeto o espacio a modelar con mayor precisión y más flexibilidad en caso de necesitar aplicar alguna transformación lineal a un subconjunto de los polígonos (rotación, traslación, reflexión, entre otros). Las mallas basadas en triángulos son particularmente útiles para describir objetos en sistemas CAD usualmente usados como planos para algún tipo de objeto concreto en 3D afecto a fenómenos físicos como la distribución de fuerzas, cuya simulación es más fiel a la realidad en un objeto descrito con el mayor nivel de detalle que sea razonable utilizar. Estas mallas también son ampliamente utilizadas en videojuegos por las mismas razones.

2.1.8. Mallas de polígonos arbitrarios

Otro uso de las mallas poligonales es en el cálculo de una solución numérica en la resolución de ecuaciones diferenciales en derivadas parciales, que describen diversos fenómenos como transferencia de calor o sonido en un espacio u objeto, las cuales no son calculables de manera exacta con un procedimiento analítico. Esta solución numérica se puede aproximar mediante el uso del *Virtual Element Method*[8] (VEM). Para el VEM son de particular importancia las mallas poligonales basadas en polígonos arbitrarios que cumplen ciertas propiedades de calidad, como tener polígonos simples, mayormente convexos, con ángulos no muy grandes ni muy pequeños, entre otras. Si la malla a utilizar parte desde una triangulación de Delaunay, algunas de estas propiedades son más fáciles de alcanzar. El uso de mallas basadas en polígonos arbitrarios permite un cálculo más rápido para el VEM y con un margen de error aceptable.

2.1.9. *Finite Element Method* (FEM) y *Virtual Element Method* (VEM)

Como fue mencionado anteriormente, el VEM[8] es un método numérico para resolver ecuaciones diferenciales haciendo uso de las mallas poligonales arbitrarias, pero antes de que existiese el VEM, existía el FEM[9, 10], estos métodos numéricos tienen el mismo objetivo, pero se diferencian en la flexibilidad permitida de los datos de entrada, siendo el FEM mucho más rígido respecto a la malla de entrada, en particular, el FEM no permite polígonos no convexos y solo acepta polígonos de un solo tipo particular como entrada, ya sean triángulos, cuadriláteros, u otros, pero siendo todos del mismo tipo, lo que no lo hace viable para algoritmos como el desarrollado en este tema de memoria.

2.1.10. Polígono simple y no simple

Un polígono se define como simple, si sus aristas forman un bucle cerrado, es decir, sin aristas internas. Por otro lado, un polígono no simple, es aquel que posee aristas internas (ver Figura 7). Estos últimos son problemáticos en prácticamente cualquier caso de uso, ya que no permiten “recorrer” los polígonos de la malla de manera regular y se tratan de eliminar de la malla de alguna manera en caso de que estén presentes.

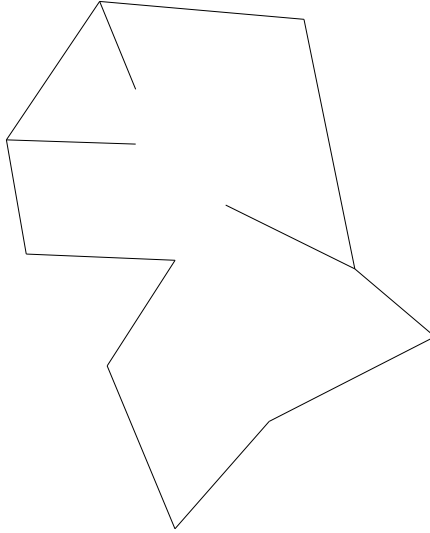


Figura 7: Ejemplo de polígono no simple

2.1.11. Cavidad

En la explicación de la solución se dará a entender como cavidad el polígono resultante de aplicar el algoritmo propuesto, es decir, dado un triángulo t_i con circuncentro p_i , la cavidad será el polígono formado por la unión de las aristas de borde del conjunto de triángulos t cuyo circuncírculo contiene al circuncentro p_i en su interior. Ver Figura 3.

2.2. Estado del arte

Existen muchos algoritmos para generación de mallas poligonales, siendo los más relevantes para este trabajo Triangle [4], Detri2 [6] y Polylla [1].

2.2.1. Triangle

El algoritmo Triangle [4] se basa en el algoritmo de Ruppert [11] y consta de 4 etapas, de las cuales las primeras 2 son exactamente las mismas que las de Ruppert, estas involucran triangular un PSLG como el de la Figura 8, y posteriormente insertar los segmentos originales de la triangulación como se ve en la Figura 9 y la Figura 10.

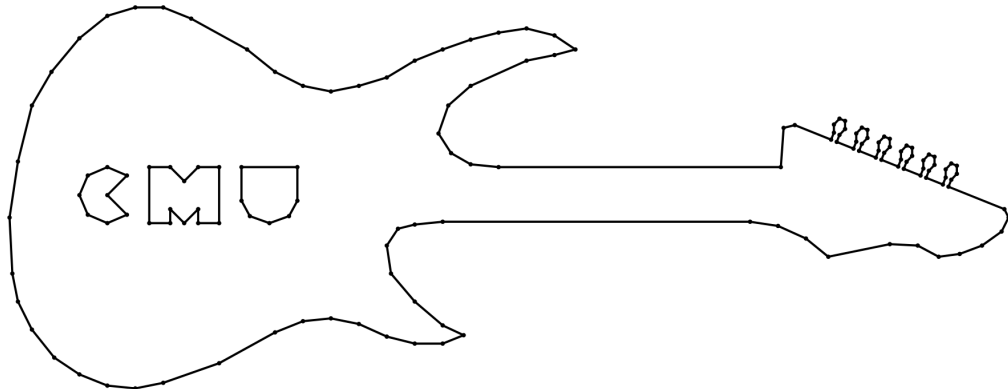


Figura 8: PSLG de entrada una guitarra electrica como se ilustra en Triangle [4]

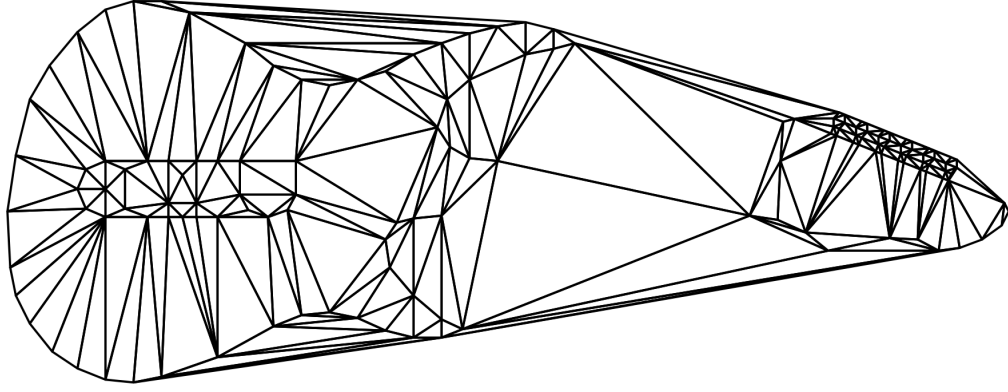


Figura 9: Triangulación del PSLG de la Figura 8 con segmentos originales faltantes [4]

Dado que esta malla no describe precisamente al polígono original del PLSG, puesto que está incluida la cápsula convexa del conjunto de puntos y otras aristas fuera del borde, la segunda etapa consta de reintroducir los segmentos originales del PLSG, esto se puede hacer de 2 maneras posibles según preferencia del usuario. La primera es insertar un nuevo vértice que corresponda al punto medio de alguno de los segmentos que no aparezcan en la triangulación anterior y usar el algoritmo incremental de Lawson [12] para obtener una nueva triangulación de Delaunay basada en la anterior con el vértice adicional. Esto genera que el segmento original se divida en 2 y la nueva triangulación podría tener el segmento original formado por estos 2 sub segmentos. En caso de que los sub segmentos aún no formen un arco en la triangulación, el proceso de insertar un vértice del punto medio se repite recursivamente para los sub segmentos hasta que el segmento original exista como una secuencia de segmentos lineales. La manera alternativa de insertar los segmentos originales, y la que se utiliza por defecto, es convertir la triangulación a una triangulación de Delaunay restringida, en la cual los segmentos originales deben aparecer. Esto se logra eliminando los triángulos que intersequen el segmento que se desea agregar y luego re triangulando las regiones a cada lado del segmento insertado, resultando en la Figura 10.

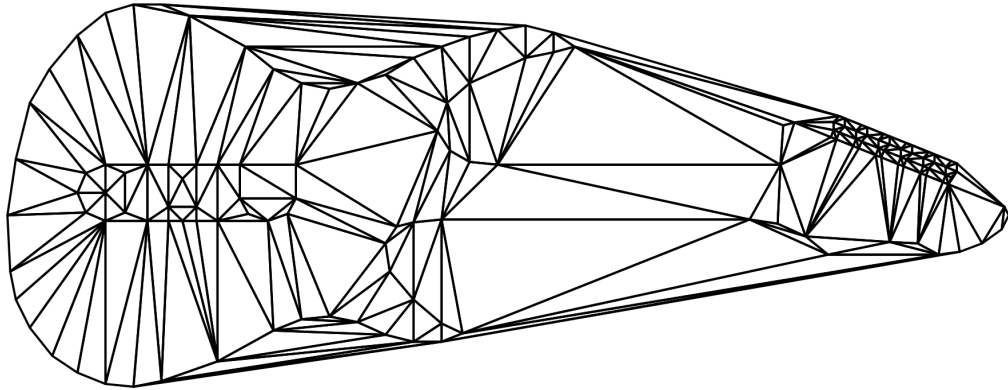


Figura 10: Triangulación restringida del PSLG de la Figura 8 [4]

La tercera etapa difiere del algoritmo de Ruppert y consiste en remover los triángulos extra que están fuera del borde definido por el PSLG original, como aquellos presentes en

zonas que originalmente eran no convexas, a las cuales Shewchuk llama concavidades, y los presentes en agujeros (como los del interior del cuerpo de la guitarra en el ejemplo). Esto se ilustra en la Figura 11.

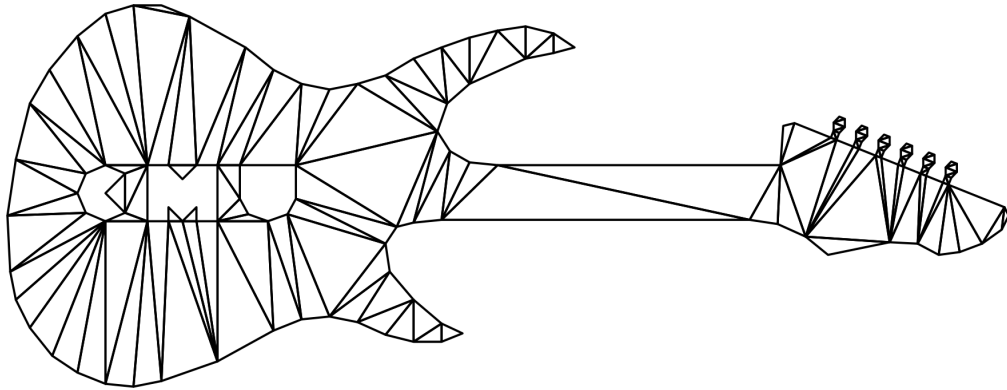


Figura 11: Triangulación restringida del PSLG de la Figura 8 con triángulos extra removidos [4]

La última etapa del algoritmo consiste en el refinamiento de la malla insertando vértices y re triangulando con el algoritmo incremental de Lawson [12] hasta que las restricciones de ángulo mínimo y área máxima de triángulo definidas por el usuario se cumplan. Esta inserción se hace siguiendo 2 reglas, la regla de *segmentos encerrados* y la de los *triángulos malos* dándole siempre prioridad a la primera:

- El *círculo diametral* de un segmento es el círculo único más pequeño que contiene el segmento como su diámetro. Un segmento se dice que está *encerrado* si un punto que no es un extremo del segmento está dentro de su círculo diametral. Cualquier segmento encerrado que aparezca se separa insertando un vértice en su punto medio. Los dos sub segmentos resultantes tienen círculos diametrales más pequeños y podrían estar o no estar encerrados. El proceso se repite hasta que no queden segmentos encerrados como se ve en la Figura 12.
- Un triángulo se dice que es *malo* dependiendo de algún criterio, por ejemplo si tiene un ángulo que es muy pequeño o un área que es muy grande para satisfacer las restricciones impuestas por el usuario. Un triángulo malo se destruye insertando un vértice en su circuncentro. Está asegurado que el triángulo malo será eliminado como se ve en la Figura 13 para mantener la propiedad de Delaunay. Si el vértice insertado encierra un segmento (como se define en la regla del círculo diametral), este será removido deshaciendo la inserción y los segmentos que encerraba se separarán según la regla del círculo diametral.

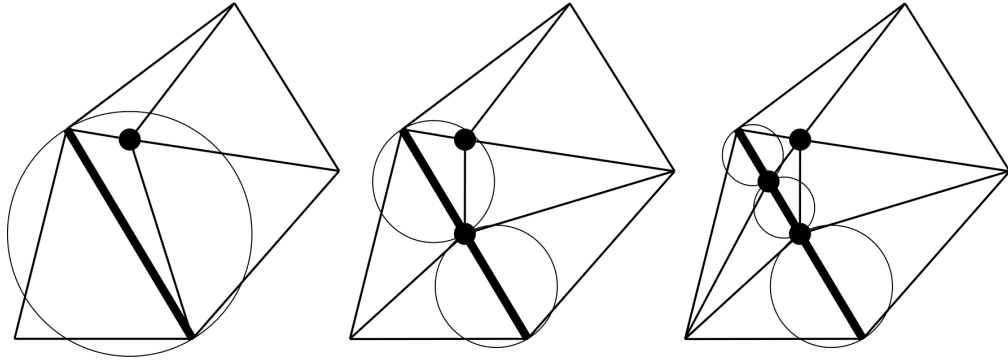


Figura 12: Segmentos divididos según su círculo diametral [4]

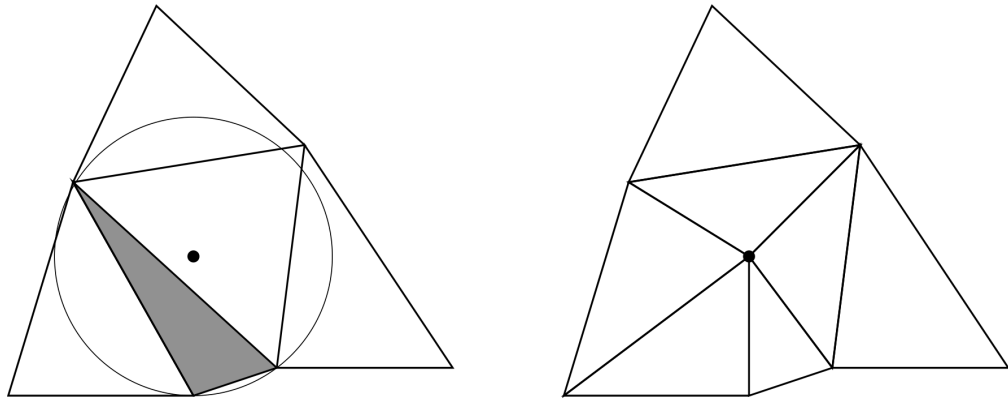


Figura 13: Triángulo separado según su circuncentro [4]

Para el ejemplo de la Figura 8, la malla resultante generada por Triangle es la que se ilustra en la Figura 14.

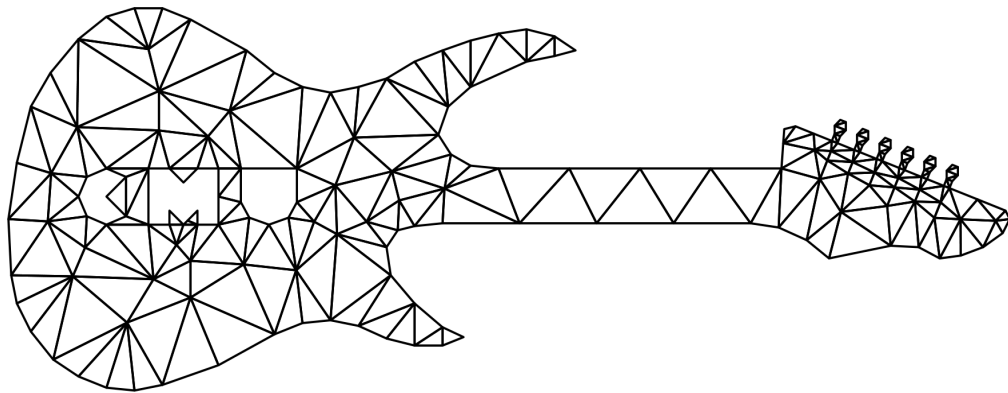


Figura 14: Malla poligonal final resultante de aplicar el algoritmo Triangle [4]

Esta última parte del algoritmo es de particular importancia, ya que el criterio del circun-círculo es análogo al de la cavidad, sin embargo, en este caso solo se utiliza para refinar la malla y re triangular las cavidades. Este trabajo de memoria busca explorar más a fondo este proceso y utilizar las cavidades para generar mallas de polígonos generales.

2.2.2. Detri2

El software Detri2[6] permite generar triangulaciones a partir de nubes de vértices aleatorios, y utilizar distintos criterios para refinar la triangulación a través de una interfaz gráfica que permite una gran variedad de opciones y resulta muy útil para generar o verificar geometrías. Además de la triangulación de un conjunto de puntos, Detri2 también permite visualizar su diagrama de Voronoi. En la Figura 4 mostrada anteriormente y la Figura 15 se puede ver una triangulación de Delaunay, su diagrama de Voronoi equivalente, y ambos superpuestos. La Figura 15 muestra un ejemplo hecho en Detri2.

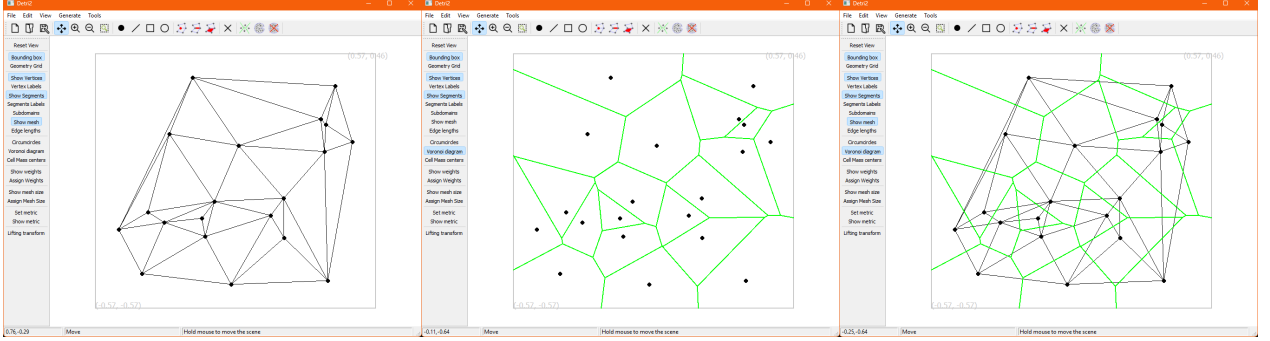


Figura 15: Triangulación de Delaunay y su Diagrama de Voronoi dual en el software Detri2

2.2.3. Polylla

Por otro lado, el algoritmo Polylla, busca generar una malla poligonal a partir de una triangulación arbitraria, usando lo que denomina como *Terminal-edge regions* o regiones de arista terminal, definidas según el *longest edge propagation path* (camino de propagación de arista más larga o *Lepp* [13]) de los triángulos, las cuales utiliza para generar una partición de la triangulación que se asemeja a un diagrama de Voronoi [14].

El *Lepp* o camino de propagación de arista más larga de un triángulo se define de la siguiente manera: Por cada triángulo t_i en cualquier triangulación Ω , el $Lepp(t_i)$ es la lista ordenada de todos los triángulos $t_0, t_1, t_2, \dots, t_{l-1}, t_l$ con $l \in \mathbb{N}$, tal que t_i es el triángulo vecino de t_{i-1} a través de la arista más larga de t_{i-1} , para $i = 1, 2, \dots, l$. Si una arista más larga es compartida por t_{l-1} y t_l esta se define como una arista terminal donde termina el Lepp y t_{l-1} y t_l son triángulos terminales. Una región de arista terminal se define como la unión de los triángulos t tal que $Lepp(t)$ termina en la misma arista terminal. En la Figura 16 se puede ver un ejemplo de región de arista terminal.

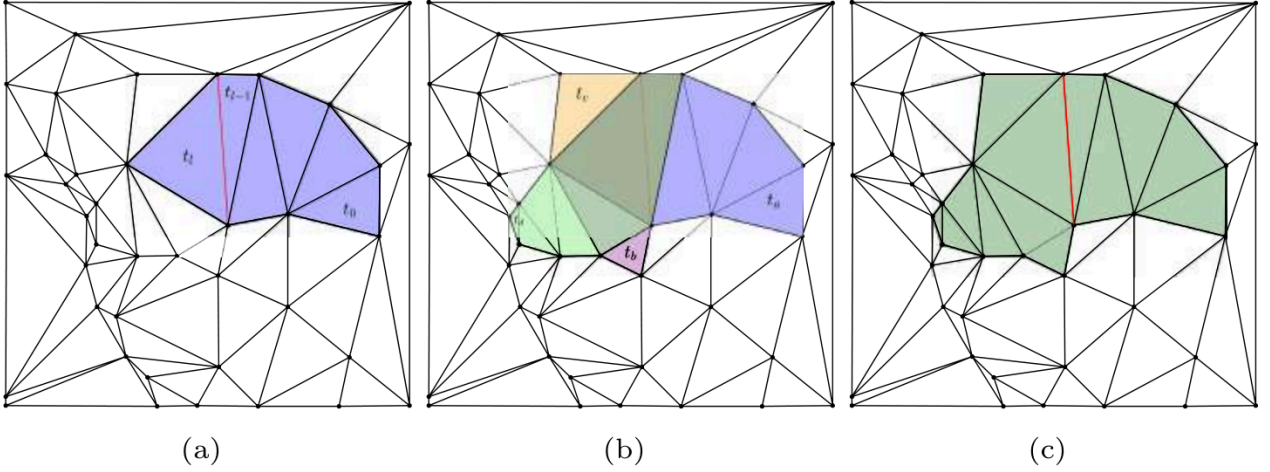


Figura 16: Región de arista terminal. a) $Lepp(t_0)$ donde la arista roja es la arista terminal. b) Cuatro Lepps con la misma arista terminal: $Lepp(t_a)$, $Lepp(t_b)$, $Lepp(t_c)$, $Lepp(t_d)$. c) Región de arista terminal generada por la unión de los Lepp de b) [1]

Además de las aristas terminales, Polylla [1] define los siguientes tipos de aristas. Dada una arista e y dos triángulos t_1 y t_2 que comparten e :

- *Frontier-edge* o Arista frontera: e no es la arista más larga ni de t_1 ni de t_2 .
- *Internal-edge* o Arista interna: e es la arista más larga de t_1 , pero no de t_2 o viceversa.
- *Boundary edge* o Arista de borde: e pertenece a un solo triángulo. Se manejan como aristas frontera.
- *Barrier edge* o Arista barrera: Arista frontera que queda adentro de una región terminal (y no es el borde).

Polylla consiste de tres fases: Primero etiqueta las aristas de la triangulación de entrada según las categorías anteriores para formar regiones terminales y además designa un *triángulo semilla* en cada región de arista terminal para construir las regiones. Luego, a partir de cada triángulo semilla, hace un recorrido en sentido antihorario o *counter clockwise* (CCW en inglés) de la región de arista terminal para encontrar aristas frontera las cuales formaran la región. Algunas regiones pueden terminar como polígonos no simples, es decir, que tienen puntos colineales o aristas que se intersecan entre sí, por lo que hace una fase de reparación donde las aristas barrera se particionan en polígonos simples. En la Figura 17 se puede ver una partición de un polígono generada por regiones de arista terminal que presenta una región con un polígono no simple en verde.

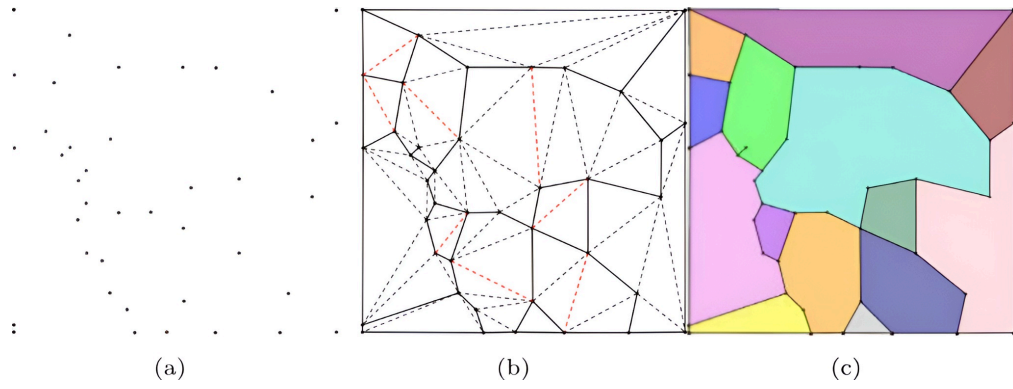


Figura 17: a) Colección de vértices aleatorios, b) Triangulación de Delaunay donde las líneas sólidas son aristas frontera, las líneas punteadas negras son aristas internas y las aristas punteadas rojas son aristas terminales. c) Partición a partir de regiones de arista terminal [1]

En la Figura 18 se puede ver una triangulación de Delaunay y la malla generada por Polylla a partir de ella.

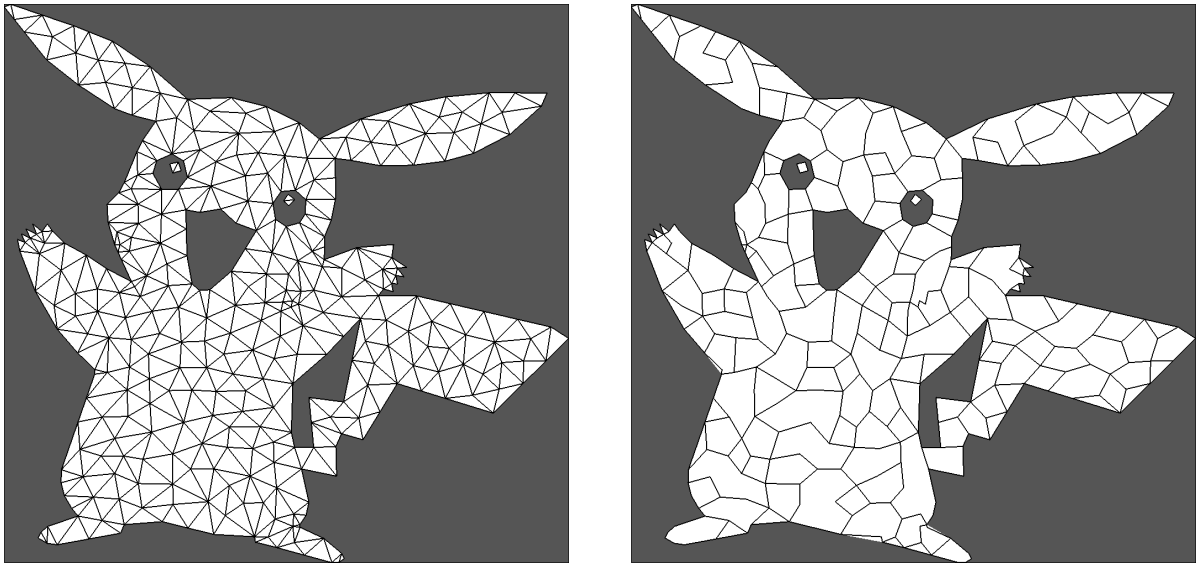


Figura 18: Triangulación de Delaunay y su malla Polylla respectiva [5]

El algoritmo Polylla destaca por sobre otros algoritmos debido a su gran simplicidad y eficiencia en comparación a algoritmos convencionales de construcción de diagramas de Voronoi restringidos, ya que toma bastante menos tiempo en construir con una cantidad de polígonos 3 veces menor y la mitad de vértices que una malla poligonal de un diagrama de Voronoi hecho a partir de la misma triangulación. A esto se le suma también la utilidad de las mallas Polylla en simulaciones de diversos fenómenos que hacen uso del VEM[8] mencionado anteriormente para encontrar una solución numérica. El algoritmo de construcción de mallas basado en cavidades se muestra como una alternativa a Polylla y mallas basadas en el diagrama de Voronoi que permite explorar el alcance de una nueva estrategia, sus propiedades generativas y de escalabilidad.

2.3. CGAL

La librería CGAL[15] (*Computational Geometry Algorithms Library*) es un proyecto de código abierto escrito en C++ que aloja una vasta colección de algoritmos geométricos y estructuras de datos eficientes y robustos. Su propósito es facilitar tareas geométricas complejas que aparecen en dominios tan variados como sistemas de información geográfica, diseño asistido por computador, biología molecular, imágenes médicas, gráficos por computador o robótica.

Dentro de sus módulos más utilizados destacan los dedicados a la generación de mallas y a las estructuras basadas en subdivisiones del plano. CGAL provee triangulaciones de Delaunay en 2D completamente funcionales, incluyendo variantes con restricciones y mecanismos de refinamiento, que facilitan producir mallas de alta calidad. A partir de estas mismas triangulaciones, la biblioteca permite obtener de forma directa el diagrama de Voronoi correspondiente, aprovechando la relación dual entre ambas estructuras. Estas capacidades hacen de CGAL una base sólida para desarrollar, comparar o extender nuevos algoritmos de generación y procesamiento de mallas.

Sin embargo, para este trabajo no se hace uso de CGAL debido a su complejidad y generalidad. Dado que CGAL es una librería enorme, no se utilizaría una cantidad considerable de sus utilidades. Prescindir de CGAL también permite mayor control sobre la implementación.

Capítulo 3

Problema

Capítulo 4

Conclusión

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Bibliografía

- [1] S. Salinas-Fernández, N. Hitschfeld-Kahler, A. Ortiz-Bernardin, y H. Si, «POLYLLA: polygonal meshing algorithm based on terminal-edge regions», *Engineering with Computers*, vol. 38, n.º 5, pp. 4545-4567, 2022, doi: 10.1007/s00366-022-01643-4.
- [2] Nü es, «A Delaunay triangulation and its Voronoi diagram». [En línea]. Disponible en: https://commons.wikimedia.org/wiki/File:Delaunay_Voronoi.svg
- [3] Accountalive, «Showing next, prev and twin of a halfedge in a DCEL». [En línea]. Disponible en: <https://commons.wikimedia.org/wiki/File:Dcel-halfedge-connectivity.svg>
- [4] J. R. Shewchuk, «Triangle: Engineering a 2D quality mesh generator and Delaunay triangulator», en *Applied Computational Geometry Towards Geometric Engineering*, M. C. Lin y D. Manocha, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 1996, pp. 203-222.
- [5] S. Salinas y R. Carrasco, *Polylla-Mesh-DCEL*. [En línea]. Disponible en: <https://github.com/ssalinasfe/Polylla-Mesh-DCEL/tree/main>
- [6] H. Si, «Detri2 is a C++ program for generating (weighted) Delaunay triangulations for (weighted) point sets in 2d.». [En línea]. Disponible en: <https://www.wias-berlin.de/people/si/detri2.html>
- [7] B. Delaunay, «Sur la sphère vide. A la mémoire de Georges Voronoï», *Известия Российской академии наук. Серия математическая*, n.º 6, pp. 793-800, 1934.
- [8] L. Veiga, F. Brezzi, A. Cangiani, G. Manzini, L. Marini, y A. Russo, «Basic Principles of Virtual Element Methods», *Mathematical Models and Methods in Applied Sciences*, vol. 23, pp. 199-214, 2012, doi: 10.1142/S0218202512500492.
- [9] V. Jagota, A. P. S. Sethi, y K. Kumar, «Finite Element Method: An Overview», *Walailak Journal of Science and Technology (WJST)*, vol. 10, n.º 1, pp. 1-8, 2013, [En línea]. Disponible en: <https://wjst.wu.ac.th/index.php/wjst/article/view/499>
- [10] A. Hrennikoff, «Solution of Problems of Elasticity by the Framework Method», *Journal of Applied Mechanics*, vol. 8, n.º 4, pp. A169-A175, dic. 1941, doi: 10.1115/1.4009129.
- [11] J. Ruppert, «A Delaunay Refinement Algorithm for Quality 2-Dimensional Mesh Generation», *Journal of Algorithms*, vol. 18, n.º 3, pp. 548-585, 1995, doi: <https://doi.org/10.1006/jagm.1995.1021>.

- [12] C. Lawson, «Software for C1 Surface Interpolation», *Mathematical Software*. Academic Press, pp. 161-194, 1977. doi: <https://doi.org/10.1016/B978-0-12-587260-7.50011-X>.
- [13] M.-C. Rivara, «New longest-edge algorithms for the refinement and/or improvement of unstructured triangulations», *International Journal for Numerical Methods in Engineering*, vol. 40, n.^o 18, pp. 3313-3324, 1997, doi: [https://doi.org/10.1002/\(SICI\)1097-0207\(19970930\)40:18%3C3313::AID-NME214%3E3.0.CO;2-%23](https://doi.org/10.1002/(SICI)1097-0207(19970930)40:18%3C3313::AID-NME214%3E3.0.CO;2-%23).
- [14] G. Voronoi, «Nouvelles applications des paramètres continus à la théorie des formes quadratiques. Premier mémoire. Sur quelques propriétés des formes quadratiques positives parfaites.», *Journal für die reine und angewandte Mathematik (Crelles Journal)*, vol. 1908, n.^o 133, pp. 97-102, 1908, doi: [doi:10.1515/crll.1908.133.97](https://doi.org/10.1515/crll.1908.133.97).
- [15] The CGAL Project, «CGAL: Computational Geometry Algorithms Library». [En línea]. Disponible en: <https://www.cgal.org/>

Anexo A

Anexo

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim admodum doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim admodum doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim admodum doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.